Laboratory Activity 1

Refactoring an Existing MERN Backend into an MVC Architecture

Name	Marzan, _____
Section	_____
Date	19 July 2026

Part 1: Project Preparation

Open the backend project created during the previous Web Programming course.

The backend project (`marzan-server`) was opened in Visual Studio Code. It is an Express.js + MongoDB (Mongoose) REST API for the BulldogEx Shop web application, which pairs with the React frontend in `marzan-client`. Dependencies were installed with `npm install`, sample products were loaded with `npm run seed`, and the server was started with `npm run dev`. The console confirmed a successful MongoDB connection and that the server is listening on port 5000.

```
> npm run dev
MongoDB connected
Server running on http://localhost:5000
```

Dependencies used: express, mongoose, dotenv, cors, bcrypt, jsonwebtoken (and nodemon for development).

[Paste screenshot here: VS Code with marzan-server open + terminal output of npm run dev]

Part 2: Analyze the Existing Structure

Identify: Routes, Database models, Server configuration, API endpoints

Routes

- `productRoutes.js` - handles all product requests under `/api/products`.
- `userRoutes.js` - handles user authentication under `/api/users`.

Database Models

- `Product.js` - Mongoose schema for shop products: name (unique URL slug), title, category, price, stock, content (description list), and image.
- `User.js` - Mongoose schema for registered users: firstName, lastName, email (unique), and password (stored as a bcrypt hash).

Server Configuration

- `index.js` - the entry point. Creates the Express app, applies the `cors()` and `express.json()` middleware, mounts the routes, connects to MongoDB, and starts the server.
- `.env` - stores environment-specific values: MongoDB connection string, salt rounds, JWT secret key, and port.
- `config.js` - loads the .env values in one central place and exports them.
- `db.js` - contains the single, reusable MongoDB connection function.

API Endpoints

Method	Endpoint	Description	Protected
GET	`/`	API status check	No

Method	Endpoint	Description	Protected
POST	`/api/users/signup`	Register a new user	No
POST	`/api/users/signin`	Log in and receive a JWT token	No
GET	`/api/users/profile`	Get the logged-in user's profile	Yes
GET	`/api/products`	Get all products	No
GET	`/api/products/:name`	Get a single product by name	No
POST	`/api/products`	Create a product	Yes
PUT	`/api/products/:name`	Update a product	Yes
DELETE	`/api/products/:name`	Delete a product	Yes

Discuss which files should belong to:

Layer	Files	Purpose
Models	`Product.js` `User.js`	Define the structure of the data and communicate with MongoDB through Mongoose.
Controllers	`productController.js` `userController.js`	Contain the business logic: read the request, call the models, and send the JSON response (e.g., product CRUD, password hashing, JWT creation).
Routes	`productRoutes.js` `userRoutes.js`	Only map an HTTP method + URL to a controller function and attach middleware where needed.
Configuration	`config.js` `db.js`	Read environment variables and establish the database connection in one place, so no other file reads `process.env` directly.
Middleware	`authMiddleware.js`	Runs before protected controllers; verifies the `Authorization: Bearer <token>` header and rejects requests with a missing or invalid token (401).

Component Relationships

A request enters through **index.js** and is passed to the matching **route**. If the route is protected, the **middleware** verifies the JWT first. The **controller** then uses a **model** to read from or write to MongoDB and returns a JSON response. Both the controllers and the middleware get their secrets (SALT, SECRET_KEY) from `config.js`, and `db.js` gets the database URL from it.

```
Client (Postman / React)
   -> index.js
   -> routes/
   -> middleware/ (protected routes only)
   -> controllers/
   -> models/  <->  MongoDB
   -> JSON response
```

Part 3: Refactoring to New MVC Structure

Create the following folders inside the component folder: config, controllers, middleware, models, routes

A `components` folder was created inside `marzan-server`, and the five MVC folders were created inside it. This separates the data layer (models), the logic layer (controllers), and the routing layer (routes), while shared configuration and middleware are kept in their own folders.

```
marzan-server/
|-- components/
|   |-- config/
|   |-- controllers/
|   |-- middleware/
|   |-- models/
```

```
|   `-- routes/
|-- node_modules/
|-- .env
|-- .env.example
|-- .gitignore
|-- index.js
|-- package-lock.json
|-- package.json
|-- README.md
|-- seed.js
`-- vercel.json
```

[Paste screenshot here: VS Code Explorer showing the components folder and its 5 subfolders]

Part 4: Moving the Existing Files

Move existing files into their appropriate folders.

File	Moved to	Reason
`config.js`, `db.js`	`components/config/`	Environment variables and database connection
`productController.js`, `userController.js`	`components/controllers/`	Request-handling / business logic
`authMiddleware.js`	`components/middleware/`	JWT verification before protected routes
`Product.js`, `User.js`	`components/models/`	Mongoose schemas
`productRoutes.js`, `userRoutes.js`	`components/routes/`	URL-to-controller mapping
`server.js`	renamed to `index.js` (root)	Main entry point of the application

Files that are not part of the application layers remain in the project root: `index.js` (entry point), `seed.js` (database seeder), `package.json`, `.env`, `.env.example`, `.gitignore`, `vercel.json` (deployment settings), and `README.md`.

[Paste screenshot here: Explorer with every folder inside components expanded]

Part 5: Creating Config File

components/config/config.js

```
require("dotenv").config();

const MONGO_DB_URL = process.env.MONGODB_CONNECTION_STRING;
const SALT = parseInt(process.env.SALT);
const SECRET_KEY = process.env.SECRET_KEY;
const PORT = process.env.PORT || 5000;

module.exports = {
  MONGO_DB_URL,
  SALT,
  SECRET_KEY,
  PORT,
};
```

`config.js` loads the `.env` file using the dotenv library and exports all environment values from one place. `SALT` is converted to a number with `parseInt` because bcrypt expects a numeric salt round, and `PORT` falls back to 5000 when it is not set.

components/config/db.js (centralized MongoDB connection)

```
const mongoose = require("mongoose");
const { MONGO_DB_URL } = require("./config");

const connectDB = async () => {
  try {
    await mongoose.connect(MONGO_DB_URL);
    console.log("MongoDB connected");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

.env

```
MONGODB_CONNECTION_STRING=mongodb://127.0.0.1:27017/bulldogex-shop
SALT=10
SECRET_KEY=********
PORT=5000
```

The `.env` file is listed in `.gitignore` so secrets are never pushed to GitHub; `.env.example` is committed as a template instead.

[Paste screenshot here: config.js code in VS Code]

Part 6: Refactoring Existing Imports and Testing

Affected Files

1. index.js - imports now point to the `components` folder.

```
const { PORT } = require("./components/config/config");
const connectDB = require("./components/config/db");
const productRoutes = require("./components/routes/productRoutes");
const userRoutes = require("./components/routes/userRoutes");
```

2. components/controllers/userController.js - uses the config file instead of `process.env`.

```
const User = require("../models/User");
const { SALT, SECRET_KEY } = require("../config/config");
```

3. components/controllers/productController.js

```
const Product = require("../models/Product");
```

4. components/middleware/authMiddleware.js

```
const { SECRET_KEY } = require("../config/config");
```

5. components/routes/productRoutes.js and userRoutes.js

```
const { ... } = require("../controllers/productController");
const { signUp, signIn, getProfile } = require("../controllers/userController");
const verifyToken = require("../middleware/authMiddleware");
```

6. seed.js

```
const connectDB = require("./components/config/db");
```

```
const Product = require("./components/models/Product");
```

[Paste screenshot here: each affected file above (add more space if needed)]

API Testing with Postman

Base URL: `http://localhost:5000`. For protected routes, add the header `Authorization: Bearer <token>` using the token returned by sign in.

#	Request	Body / Header	Result
1	GET `/`	-	200 - `BulldogEx Shop API is running`
2	POST `/api/users/signup`	`{"firstName": "Juan", "lastName": "Cruz", "email": "juan@nu.edu.ph", "password": "pass123"}`	201 - user created
3	POST `/api/users/signin`	`{"email": "juan@nu.edu.ph", "password": "pass123"}`	200 - returns JWT token
4	GET `/api/users/profile`	Bearer token	200 - profile (no password)
5	GET `/api/products`	-	200 - list of 8 products
6	GET `/api/products/hoodie-jacket`	-	200 - single product
7	POST `/api/products`	Bearer token + `{"name": "test-item", "title": "Test Item", "category": "Bags", "price": 499, "stock": "In stock"}`	201 - product created
8	PUT `/api/products/test-item`	Bearer token + `{"price": 399}`	200 - price updated to 399
9	DELETE `/api/products/test-item`	Bearer token	200 - Product deleted
10	POST `/api/products`	No token	401 - No token provided

[Paste screenshot here: Postman result for each request (one screenshot per test)]

Conclusion

After refactoring, all imports resolved correctly and the server started without errors. The API was verified using Postman: user registration, login, and the protected profile route worked, and full CRUD operations on products succeeded. Protected routes correctly rejected requests without a valid token (401). This confirms that moving the files into the MVC structure under `components/` and centralizing environment variables in `config.js` kept the application fully functional while making the codebase more organized and maintainable.